

CS 4610 HW 1

Answer the following questions from the Textbook. 10 points for each question

- 1. Are there dependencies between any of the security concepts that we covered? For example, is authentication required for authorization? Why or why not?**

Authentication is generally a prerequisite for authorization because a system cannot accurately determine what a user is allowed to do until it has verified who that user is. For instance, a corporate database must first authenticate an employee's identity before it can authorize them to access sensitive payroll records based on their specific role.

Accountability is strictly dependent on authentication, where you cannot hold a specific user responsible for an action in an audit log if their identity was never verified.

Confidentiality relies on authorization to ensure that data remains secret by only allowing those with the proper permissions to view it.

- 2. Why is two-factor authentication better than one-factor authentication?**

2FA is superior to 1FA because it implements the principle of defense in depth by requiring proof from two distinct categories of identity verification. In a 1FA system, which usually relies solely on something you know like a password, an attacker who successfully phishes a credential or uses a keylogger gains immediate and total access.

By contrast, 2FA requires a second factor from a different category, such as something you have (a physical token or OTP card) or something you are (biometrics). This significantly raises the bar for an attacker, where even if they steal a user's password, the account

remains protected because the attacker does not possess the user's physical device or biological traits.

This effectively mitigates the inherent weaknesses of passwords, such as reuse and low entropy, providing a much higher level of confidentiality and account security.

- 3. A Trojan horse is a program that advertises that it does one thing, but really does something that's malicious. For example, a computer chess program that deletes the contents of the user's hard drive when the user loses the game is a Trojan horse. How would the Bell-LaPadula model prevent an attacker that only has unclassified access to a system from obtaining top secret information even if the attacker is able to trick his boss (who has top secret clearance) to run a Trojan horse?**

The Bell-LaPadula model prevents this data leak through the No Write Down rule. When the boss runs the Trojan horse, the program operates with the boss's high-level privileges. While the Trojan would be permitted to read Top Secret information because it shares the boss's clearance, the Star Property strictly forbids that program from writing any data to an object at a lower security level, such as an Unclassified file.

Since the attacker only has Unclassified access, the Trojan has no way to drop the stolen information into a location where the attacker is permitted to read it. By enforcing this one-way flow of information, the Bell-LaPadula model ensures that even if a high-clearance user is tricked into running malicious code, the system's confidentiality remains intact, and the information remains trapped at the higher level.

- 4. State the difference between confidentiality and data integrity in three sentences or less.**

Confidentiality ensures that information remains secret and is only accessible to authorized users, primarily protecting against unauthorized viewing or eavesdropping. Data integrity ensures that

information remains accurate and has not been tampered with or modified by unauthorized parties during transmission or storage. While confidentiality prevents an attacker from seeing the data, integrity prevents them from changing it.

5. What happens if a client connects to SimpleWebServer, but never sends any data and never disconnects? What type of an attack would such a client be able to conduct

If a client connects to SimpleWebServer but never sends data, the server will stall indefinitely at the `br.readLine()` instruction inside the `processRequest` method. Because the code is single-threaded and uses blocking I/O, the execution flow is trapped waiting for a line of text that never arrives, which prevents the server from returning to the `accept()` call to handle other users. This scenario allows even a single attacker to conduct a Denial of Service (DoS) attack, effectively rendering the server unavailable to all legitimate traffic by occupying its only thread of execution and exhausting the system's availability.

6. HTTP supports a mechanism that allows users to upload files in addition to retrieving them through a PUT command.

a. What threats would you need to consider if SimpleWebServer also had functionality that could be used to upload files?

If SimpleWebServer were extended to support file uploads via the PUT command, some threats I can think of would emerge. The most immediate risk is System Compromise through Path Traversal, where an attacker provides a filename like `../../etc/shadow` to overwrite sensitive system files and gain root access.

Another major threat is the Execution of Malicious Code, such as a Web Shell or Trojan, which could be uploaded and then triggered by the attacker to take complete control of the server.

The system also faces an Availability threat through Disk Space Exhaustion, a form of Denial of Service where an attacker uploads

massive or infinite amounts of data to fill the server's hard drive and crash the application.

Without proper Access Control, any anonymous user could upload illegal or sensitive content, leading to a breach of the intended security policy.

b. For each of the specific threats you just listed, what types of security mechanisms might you put in place to mitigate the threats?

To prevent path traversal and overwriting, the programmer must use strict Input Validation and Sanitization to strip dangerous characters from filenames and ensure all uploads are restricted to a Sandboxed directory.

To prevent the execution of malware, the system should follow the principle of Least Privilege by disabling execute permissions on the upload folder and using On-Access Anti-Virus scanning to check files immediately upon receipt.

Availability can be protected by enforcing strict File Size Limits and User Quotas to prevent a single user from exhausting system resources.

Authentication and Authorization must be required for all put operations to ensure only verified users can modify the server's state, satisfying the goals of Accountability and Integrity.

7. Rewrite the `serveFile()` method such that it imposes a maximum file size limit. If a user attempts to download a file that is larger than the maximum allowed size, write a log entry to a file called `error_log` and return a "403 Forbidden" HTTP response code (5 points). What happens if an attacker tries to download `/dev/random` after you have made your modification?

Modified `serveFile()` Method:

```

public void serveFile(OutputStreamWriter osw, String pathname) throws
Exception {
    // Define a maximum file size limit
    final long MAX_FILE_SIZE = 1048576;

    // path sanitization
    if (pathname.charAt(0) == '/') pathname = pathname.substring(1);
    if (pathname.equals("")) pathname = "index.html";

    File targetFile = new File(pathname);

    if (!targetFile.exists() || !targetFile.isFile()) {
        osw.write("HTTP /1.0 404 Not Found\n\n");
        return;
    }

    // max file size
    if (targetFile.length() > MAX_FILE_SIZE) {
        try (FileWriter fw = new FileWriter("error_log", true);
             PrintWriter pw = new PrintWriter(fw)) {
            pw.println(new java.util.Date() + " - ERROR: Access denied
to " + pathname +
                        ". Size (" + targetFile.length() + ") exceeds
limit.");
        }

        // return 403
        osw.write("HTTP /1.0 403 Forbidden\n\n");
        return;
    }

    // If size is valid, serve the file
    try (FileReader fr = new FileReader(targetFile)) {
        osw.write("HTTP /1.0 200 OK \n\n");
        int c;
        while ((c = fr.read()) != -1) {
            osw.write((char) c);
        }
    }
}

```

```
    } catch (Exception e) {  
        osw.write("HTTP /1.0 500 Internal Server Error\n\n");  
    }  
}
```

Analysis:

If an attacker attempts to download `/dev/random` after this modification, the server will likely fall victim to a Denial of Service (DoS) attack.

In linux systems, `/dev/random` is a special character device that produces a continuous, infinite stream of random data. When the server calls `targetFile.length()` on this device, the operating system typically returns a size of 0 bytes. Because 0 is less than the `MAX_FILE_SIZE` limit, the security check will pass, and the server will enter the while loop to read the file.

Since `/dev/random` never reaches an End of File state, the loop will never terminate, causing the server thread to hang indefinitely while consuming CPU and memory resources to stream infinite data. This is a failure in Input Validation, as the code assumes it is interacting with a regular file rather than a system device, ultimately compromising the Availability of the server.